

Slide 1: Capa (An Experimental Evaluation of TEE Technology)

- **Tempo estimado:** 1,5 minutos
- **Objetivo:** Apresentar o tema central, a equipe do artigo e introduzir o conceito de "dados em uso" e a motivação inicial.
- **Fala sugerida:**

"Olá a todos. Meu nome é Nailson Francisco da Silva Chagas e hoje, como atividade da disciplina de Segurança Computacional, vou apresentar uma análise profunda do artigo '*An experimental evaluation of TEE technology: Benchmarking transparent approaches based on SGX, SEV, and TDX*', conduzido por Luigi Coppolino e equipe.

Tradicionalmente, a segurança de dados lida muito bem com a criptografia de discos (dados em repouso) e conexões de rede (dados em trânsito). No entanto, para fechar o que chamamos de 'ciclo da confiança', precisamos proteger os dados durante o processamento, quando eles estão abertos na memória. É aqui que entram os *Trusted Execution Environments*, ou TEEs. O pioneiro foi o Intel SGX, que protegia os dados, mas exigia uma refatoração complexa do código das aplicações. Como veremos, isso motivou a criação de camadas intermediárias, os *runtimes* (como Gramine e Occlum), e tecnologias baseadas em Máquinas Virtuais, como o AMD SEV e o novíssimo Intel TDX. Vamos entender o quão bem essas opções funcionam na prática."
- **Transição:** "Para contextualizar o problema que os autores resolveram, vamos ao segundo slide."

Slide 2: Contexto e Motivação

- **Tempo estimado:** 1,5 minutos
- **Objetivo:** Explicar a necessidade do paradigma *Lift-and-Shift* e a lacuna da literatura que a pesquisa buscou preencher.
- **Fala sugerida:**

"O grande motor dessa pesquisa é a área de *Confidential Computing*, que busca garantir o isolamento e a privacidade dos dados de ponta a ponta na nuvem. Porém, reescrever código legado para rodar em hardware seguro é caro e inviável para muitas empresas.

Por isso, o mercado busca soluções de *Lift-and-Shift*: a capacidade de pegar um banco de dados ou aplicação web antiga e rodá-la num ambiente seguro (TEE) sem modificar absolutamente nada no código original. Apesar da indústria ter avançado nisso com abordagens em nível de processo (SGX) e de VM (SEV e TDX), havia uma enorme lacuna na literatura científica: faltavam comparações empíricas exaustivas de desempenho e uso de recursos entre essas arquiteturas. E mais importante, o Intel TDX foi lançado recentemente, tornando este estudo uma das primeiras avaliações reais dessa tecnologia."
- **Transição:** "Mas antes de olharmos para os resultados, precisamos entender uma diferença fundamental de arquitetura. Observem as imagens no próximo slide."

Slide 3: Modelos de Confiança: Processos vs VMs

- **Tempo estimado:** 2 minutos
- **Objetivo:** Usar as Figuras 1 e 2 do slide para explicar as diferenças da Base de Computação Confiável (TCB) entre SGX e SEV/TDX.
- **Fala sugerida:**

"Aqui temos duas figuras fundamentais extraídas do artigo. Na figura 1 (acima) e na figura 2 (abaixo), vemos as diferenças cruciais na fronteira de confiança, que chamamos de TCB. Olhando para o modelo *Process-based* (o Intel SGX) do lado esquerdo da Figura 2, o nível de paranoia é máximo. Apenas a CPU e o código do *enclave* são confiáveis. O Sistema Operacional hospedeiro e o Hipervisor são considerados inimigos. Se o SO for invadido, o enclave continua seguro. No entanto, por causa disso, a aplicação não pode falar diretamente com o hardware. Já do lado direito, no modelo *VM-based* (usado pelo AMD SEV e Intel TDX), toda a memória da máquina virtual é criptografada com uma chave em hardware. A Base de Confiança agora inclui o Sistema Operacional Convidado (Guest OS). Você confia no seu SO, mas não no Hipervisor do provedor da nuvem. Isso reduz ligeiramente a segurança frente ao SGX, mas ganha imensamente em compatibilidade."
- **Transição:** "Com essas arquiteturas definidas, como os autores montaram o experimento para testá-las de forma justa?"

Slide 4: Metodologia Experimental

- **Tempo estimado:** 2 minutos
- **Objetivo:** Explicar a Figura 4 do slide e destacar a normalização matemática essencial para os testes.
- **Fala sugerida:**

"Neste diagrama do ambiente experimental (Figura 4), podemos ver como os autores isolaram as variáveis. Eles usaram servidores dedicados ('bare metal') ao invés da nuvem pública para evitar ruídos de outros clientes (noisy neighbors). O Intel Xeon Platinum de 4ª geração hospedou o TDX, o SGX e os testes nativos. Já o AMD EPYC de 3ª geração hospedou a VM com SEV. Aqui há um detalhe importantíssimo: o processador AMD tinha um clock base mais baixo e era cerca de 40% mais lento do que o Intel em processamento bruto. Se os autores simplesmente rodassem os testes, o AMD SEV pareceria ineficiente. Portanto, eles aplicaram uma normalização rigorosa baseada em benchmarks de CPU, isolando exclusivamente o custo (overhead) gerado pela camada de criptografia. Para os testes, foram escolhidas três categorias de aplicações: Banco de dados em memória (Redis e Vault); I/O e Rede (NGINX e NodeJS); e Processamento de CPU (PyTorch e TensorFlow)."
- **Transição:** "Vamos analisar os resultados, começando com as aplicações que demandam muita memória RAM."

Slide 5: Performance: Redis e Vault

- **Tempo estimado:** 2 minutos

- **Objetivo:** Usar os gráficos (Fig 5 e 6) para ilustrar a eficiência do TDX e o gargalo estrutural do SGX.

- **Fala sugerida:**

"Aqui temos o comportamento das arquiteturas rodando Redis (Figura 5 à esquerda) e Vault (Figura 6 à direita). Em ambos os gráficos, o eixo horizontal é o fluxo de dados (Throughput) e o vertical é o tempo de atraso (Latency).

Observando a linha preta (ambiente Nativo, sem proteção) e a linha vermelha (Intel TDX), vemos que elas andam quase juntas. O TDX teve um desempenho espetacular, atingindo altos *throughputs* com latência baixíssima. A linha verde, do AMD SEV, mostra um overhead real de cerca de 22% em relação ao TDX, considerando a normalização.

O grande destaque negativo são as linhas amarela e azul (Gramine e Occlum no SGX). A latência dispara quase imediatamente, formando essa reta vertical no gráfico. Por que isso ocorre? O SGX tem uma área rígida de memória chamada EPC (*Enclave Page Cache*). Como o Redis pede muita memória, o SGX esgota o EPC e entra num ciclo violento de paginação (swapping), tendo que criptografar e descriptografar dados entre a RAM e o enclave continuamente. É um desastre de performance."

- **Transição:** "Se o SGX sofre com a memória, o cenário se repete quando precisamos enviar muitos dados pela rede?"

Slide 6: Performance: NGINX e NodeJS

- **Tempo estimado:** 2,5 minutos
- **Objetivo:** Explicar (Fig 9 e 10) o custo massivo do 'Context Switching' no SGX e a estabilidade das VMs.

- **Fala sugerida:**

"Para cargas de I/O, ou seja, entrada e saída de rede e disco, temos o servidor web NGINX (Figura 9) e o NodeJS (Figura 10).

As abordagens de VM (TDX em vermelho e SEV em verde) brilham novamente. O TDX manteve altíssima escalabilidade sob forte carga de rede.

Mas olhem novamente para o SGX (Gramine e Occlum). No NodeJS (gráfico da direita), eles simplesmente pararam de funcionar com poucas requisições simultâneas e os pesquisadores tiveram que diminuir drasticamente o estresse do teste só para conseguir rodá-los. A justificativa é técnica: sempre que uma aplicação web recebe um pacote de rede, ela precisa fazer uma Chamada de Sistema (Syscall). Lembra que o SGX não confia no SO? Isso significa que para toda Syscall, a CPU precisa suspender a execução segura, sair do enclave (*Enclave Exit*), pedir ao SO para lidar com a rede, e depois voltar para o enclave. Esse constante *Context Switching* arruína a performance de I/O."

- **Transição:** "Parece que o SGX não compensa, certo? Mas as coisas mudam de figura no próximo slide, onde entramos no mundo da Inteligência Artificial."

Slide 7: Performance: PyTorch e TensorFlow

- **Tempo estimado:** 2 minutos
- **Objetivo:** Usar a Figura 7 para mostrar onde o SGX domina e por que o isolamento estrito ajuda em IA.
- **Fala sugerida:**

"Aqui temos os resultados de inferência matemática pura com PyTorch. O eixo vertical indica o tempo de inferência; logo, barras menores são melhores.

A surpresa: reparem na barra amarela, que é o Gramine-SGX. Em vários algoritmos, ele foi mais rápido que o Intel TDX (barra vermelha) e o AMD SEV (barra verde) listrado. Como isso é possível se acabamos de ver que o SGX tem uma performance horrível em rede?

A resposta é a natureza do algoritmo. Na inferência de redes neurais, o dado entra na memória uma vez só, e o processador passa segundos focados exclusivamente em multiplicação de matrizes puras. Sem acessos repetidos à rede, não há *Syscalls*. Sem *Syscalls*, não há saídas do enclave. Sendo assim, a matemática pura roda livremente de forma super protegida. Nesses casos, o modelo de processos (SGX) é excelente, pois te entrega o maior isolamento e segurança de hardware possível (menor TCB) sem sacrificar velocidade."
- **Transição:** "Esses resultados de tempo são importantes, mas na vida real, ciclos de processador custam dinheiro."

Slide 8: Consumo de Recursos e Custos

- **Tempo estimado:** 2,5 minutos
- **Objetivo:** Relacionar performance computacional com faturamento em provedores de nuvem (Fig 11 e 12).
- **Fala sugerida:**

"No topo (Figura 11), temos a utilização média de CPU. As barras vermelhas do TDX mostram que ele consome ciclos de CPU de forma muito semelhante ao ambiente nativo (barra preta). Já as barras das tecnologias baseadas em SGX chegam a estourar os limites para compensar o constante gerenciamento do EPC e saídas do enclave.

Como 'CPU = Dinheiro' na nuvem, isso afeta diretamente o custo, como vemos nos gráficos da Figura 12. Nesses gráficos de custo em dólares por hora na Azure (onde mais baixo é melhor), percebemos que o SGX (linhas azul e laranja) é uniformemente a tecnologia mais cara do mercado. Para atingir a mesma quantidade de requisições de uma VM normal, você precisa provisionar máquinas SGX maiores e com muito mais núcleos.

Por outro lado, o AMD SEV (linha verde) se destaca como a opção comercial mais barata, enquanto o TDX (linha vermelha) oferece um balanço muito saudável entre performance premium e custo razoável."
- **Transição:** "Como o SGX tem casos de uso muito valiosos em IA, uma pergunta comum é: qual runtime usar para ele?"

Slide 9: Comparação: Gramine vs Occlum

- **Tempo estimado:** 1,5 minutos
- **Objetivo:** Usar a Figura 3 (diagramas de blocos) para justificar a clara superioridade do runtime Gramine.
- **Fala sugerida:**

"Ao usar o SGX para rodar aplicações sem reescrever código, precisamos de um Library OS (*LibOS*). As duas opções mais maduras são o Gramine (diagrama da esquerda) e o Occlum (diagrama da direita).

Os dados deste artigo foram definitivos: o Gramine esmagou o Occlum em performance e consumo de recursos em quase todos os testes.

Por que isso acontece? O Gramine (à esquerda) adota uma arquitetura leve: ele tenta agrupar *syscalls* para evitar saídas desnecessárias e só aloca a memória quando ela é estritamente necessária (*lazy allocation*). Já o Occlum (à direita), desenhado em uma arquitetura multiprocessos complexa, prioriza um isolamento interno chamado de SFI (*Software Fault Isolation*). Ele exige verificações binárias estritas. Essa paranoia extra cria um peso computacional que, para a maioria das empresas, não justifica a enorme perda de performance."
- **Transição:** "Com todas essas informações, vamos resumir as recomendações pragmáticas do artigo."

Slide 10: Trade-offs e Recomendações Práticas

- **Tempo estimado:** 1,5 minutos
- **Objetivo:** Sintetizar os resultados práticos usando a tabela do slide.
- **Fala sugerida:**

"Chegamos à nossa tabela de decisão, que resume as recomendações.

 - **Intel SGX (Process-based):** Deve ser a escolha apenas quando o isolamento criptográfico máximo é a exigência central e você está rodando tarefas matemáticas intensas (como HPC e Inteligência Artificial) ou protegendo chaves de altíssimo valor. A penalidade em I/O e custo inviabiliza outros usos.
 - **Intel TDX (VM-based):** É excelente. Com isolamento moderado (protege a VM inteira), facilita o uso e entrega uma performance impecável para aplicações de banco de dados, microsserviços legados e rede pesada.
 - **AMD SEV (VM-based):** É extremamente similar ao TDX no conceito e facilidade, mas brilha no aspecto financeiro, sendo a opção de mais baixo custo para criar clusters na nuvem nativa."
- **Transição:** "Para fechar a nossa análise."

Slide 11: Conclusões Finais

- **Tempo estimado:** 1 minuto
- **Objetivo:** Fixar as lições centrais do paper para a audiência.

- **Fala sugerida:**

"Em suma, as conclusões principais dos autores são claras:

Para o mundo real de TI (servidores web, bancos de dados, microsserviços), abordagens baseadas em VM (TDX e SEV) são muito superiores às abordagens baseadas em processo (SGX) devido à total transparência e alta performance de entrada e saída.

A grande estrela e novidade do estudo foi provar que o Intel TDX superou o SEV (mesmo após normalização matemática), mostrando que a tecnologia chegou ao mercado altamente eficiente. Apesar das desvantagens, o estudo resgatou o valor do SGX como padrão ouro para o nicho de IA. E a mensagem final é muito otimista: a era da *Confidential Computing* chegou a um nível de maturidade em que já é técnica e comercialmente viável para sistemas em produção em larga escala."

- **Transição:** (Pausa breve de 2 segundos. Olhe para os colegas ou câmera).

Slide 12: Dúvidas?

- **Tempo estimado:** 0,5 minutos
- **Objetivo:** Agradecimento e abertura formal para debate.
- **Fala sugerida:**

"Assim, encerro minha apresentação sobre essa avaliação empírica das tecnologias TEE. Muito obrigado a todos pela atenção. Fico agora à disposição do professor e dos colegas para quaisquer perguntas ou debates sobre o assunto!"